2010 2nd Conference on Environmental Science and Information Application Technology

GPU-Accelerated Parallel Algorithms for Map Algebra

Jianbo Zhang, Wenxin Yang, Jing Sun and Yonghong Lv
Department of Software Engineering
Faculty of Information Engineering, China University of Geoscience
Wuhan, China
Email: zjb_tigers@126.com, ywx19880209@163.com,
jing8759@163.com, lvyonghong1025@163.com

***Abstract*—Aiming at the low efficiency when traditional realization methods of map algebra apply to calculations for gigantic raster data, this paper maps the traditional serial algorithms to GPU parallel processing architecture on a new parallel programming model of GPU named Compute Unified Device Architecture. The paper also aims to discuss the realization mechanism surrounding parallel mapping methods from traditional serial algorithms to parallel algorithms and adaptive-parameter adjustments on computer graphic processor resources.**

***Keywords*-Map Algebra, GPU, CUDA, Parallel Computation**

I. INTRODUCTION

Map Algebra is a theory and methodology which provides solutions both for visualization of geographic information graphic symbol and spatial analysis. As the core technology of raster data spatial analysis, it is widely used in meteorological monitoring, land use, agricultural production analysis and other professional fields. Currently, the main problem that based on spatial analysis of raster data is no longer the lack of data sources since the precision of global measurement accuracy has reached 10-9, but how to use or improve the existing method of map algebra to extract more rich and useful information and implement the dynamic analysis for multi-dimensional and multi-source spatial information data under the global environment.

With the incredible advancement of GPU parallel processing capability and the rapid development of program ability in recent years, GPGPU(General Computation on GPU) has gradually became a hot research field. In some specific areas of computing power-sensitive, GPU has been proved to be more competitive than the single CPU.

Considering the arithmetic operators of map algebra have the following characteristics: based on grid point set, processing flow is relatively fixed, and data processing has inherent parallelism. The new GPU parallel programming model named CUDA is selected as technique supports for optimization of spatial analysis algorithms in this paper.

II. RELATED WORK

Map Algebra is a raster modeling language that proposed by Tomlin et al.[1]. Tomlin's map algebra defines four types of higher-order functions for coverage, which are: Local functions, Focal functions, Zonal functions and Global functions. The differences of these functions are reflected in the different data range of object operation: Local functions are based on a single pixel, Focal functions are based on cell sets of pixel neighborhood, Zonal functions are based on cell sets of pixel classified region and Global functions are based on all pixel data of the grid.

In recent years, many researchers focus on extending or improving the Tomlin's map algebra theory. Pullar[2] develops MapScript which is a language that allows control structures and dynamic models to be incorporated into map algebra. Ostlander[3] suggests map algebra could be embedded in a web service. Mennis et al.[4] propose an extension of map algebra for spatio-temporal data handling. Frank[5] discusses how map algebra can be formalized in a functional programming context and how this approach provides supports both for spatial and spatio-temporal operations. Gilberto et al.[6] bring forward a universal map algebra theory for both vector and raster data operations.

On the other hand, the research fields on GPU are various. Yacine Amara et al.[7] present a GPU architecture named Tile-Load-Map for the real-time visualization of wide textured terrains created with arbitrary meshes. Eladio Gutierrez et al.[8] discuss their experience in porting an application kernel to the GPU and all classes of design decisions in order to obtain maximum performance. S.S.Stone et al.[9] describe the acceleration of advanced magnetic resonance imaging (MRI) reconstruction algorithms on graphics processing units which can achieve twenty-one times speedup. Weiguo Liu et al.[10] present a new approach to accelerate molecular dynamics simulations with inexpensive commodity graphics hardware by using the new CUDA programming interface to implement a new parallel algorithm.

Altogether, most study on the map algebra are surrounding the extensions of Tomlin basic theory and the spatio-temporal data model, but less involved on algorithms' optimization. And most applications on GPU take advantage of its parallel computing characteristics and propose the new idea of transplanting the traditional serial computing model to the parallel programming model with the professional practical problems. But the research of implementing raster data analysis algorithms on GPU is rare.

978-1-4244-7388-5/10/$26.00 ©2010 IEEE

III. PARALLEL PROCESSING ARCHITECTURE OF MAP ALGEBRA ALGORITHM BASED ON GPU

A. Parallel Operator Selection of Map Algebra

Local functions of map algebra implement the same operations on a single cell. Such arithmetic operators have the characteristics of weak data relation and modularization. In this paper, we select local functions as the research objects. According to the different number of parameters, those functions can be divided into four categories:

1) 0-ary:

There are two functions Normal and Rand, which can be used to generate random datasets.

2) Unary:

For single element computation of the raster data, the operation object may be the all element or the analysis window element set. The function is defined as: OutGrid=Abs(-5 + ingrid1). Such mathematical functions can be divided into four categories: general mathematical functions (e.g. Exp, Log2), bit operation functions, triangle mathematical functions and logic operation functions(e.g.Test, IsNull).

3) Binary:

The operation function is the same as the single variable function except the calculation working on the overlap region of two raster data.

4) N-ary:

They can be used to statistic the element of corresponding positions between the multi-layer raster data. The operation object is a list of raster data of the same resolution. Such functions can be defined as: OutGrid=Func([Grid1],..., [Gridn]) and can be divided into five categories:

a) Multiple raster cell statistics

There are ten arithmetic operators: Mean, Majority, Minority, Maximum, Minimum, Range, Median, STD, Sum and Variety.

b) Multi-dimensional grid merge

The Combine function unifies value attribute table of multi-layer raster data and merges them to the single value of multi-pixel raster data.

c) The emergence frequency statistics

These functions take the first raster data as the statistical benchmark and count the element appearance frequency at the corresponding positions of the other raster datasets. Such functions include three operators: Greater Than, Equal To and Less Than.

d) Evaluation of the multi-grid value

There are two operators Popularity and Rank. They count the pixel value of corresponding positions of the multiple grids in a certain standard.

e) Determine the position of extremum

They are used to determine the grid serial number with a maximum or minimum value at corresponding positions of the multiple grids. They include two operators LPos and UPos.

B. CUDA Architecture Research

With the birth of technology named Compute Unified Device Architecture(CUDA) and release of Geforce 8800 series of graphics cards, GPU programming mode has been reformed incredibly. Developers implement programming in virtue of graphics pipeline API (for example OpenGL) no longer. CUDA is a new hardware and software cooperative-work architecture in which more transistors are devoted to GPU for data processing rather than data caching and flow control. It views GPU as a data-parallel computing device and treats with parallel computations without the need of mapping them to complex graphics API.

C. Study Memory Access Mechanism of CUDA and Optimize the Memory Utilization

In the CUDA framework, GPU has its own static memory, on-chip shared memory, as well as Streaming Multiprocessor (SM) made of a series of streaming processing units. Static memory can be categorized as read-only texture memory, read-and-write global memory and the constant read-only memory. Data stored in SRAM can be accessed by all Streaming Multiprocessors but with speed penalty. In comparison with SRAM, the access speed of shared memory on each streaming multiprocessor is much faster. However, they could only be accessed by their bounded SM.

Taking full advantage of on-chip memory is critical to improve the efficiency of parallel algorithms on map algebra arithmetic operators. We could take different approaches to optimize all types of storage accordingly:

1) Read-only Texture Memory:

Read-only Texture Memory could be used to store frequently used two-dimensional raster data because its read-write access to 2D constant data is hardware-optimized in order to minimize duplication of data access during the execution of algorithms.

2) Constant Read-only Memory:

Constant Read-only Memory could be used to store frequently used arithmetic constant data for the access latency to this memory is as low as to registers.

3) Read-and-Write Shared Memory:

Read-and-Write Shared Memory could be used to store intermediate calculating results of threading blocks. This could avoid defects such that slow data exchanges through video memory between thread modules. It can improve the use efficiency of on-chip memory.

4) Global Memory:

Global Memory can be used to store input or output data which are not frequently accessed.

5) Registers:

Registers could be used to store results of threads during calculations. It's extremely fast.

D. Study CUDA Threading Model and Optimize the Available Calculating Resources

In the CUDA programming model, all the tasks that running on GPU are done by threads in a series of assigned

blocks by users. Threads form the block in form of warp, and all of the blocks compose a parallel computing grid in form of array structure. The following figure shows the 3-level thread model of CUDA.

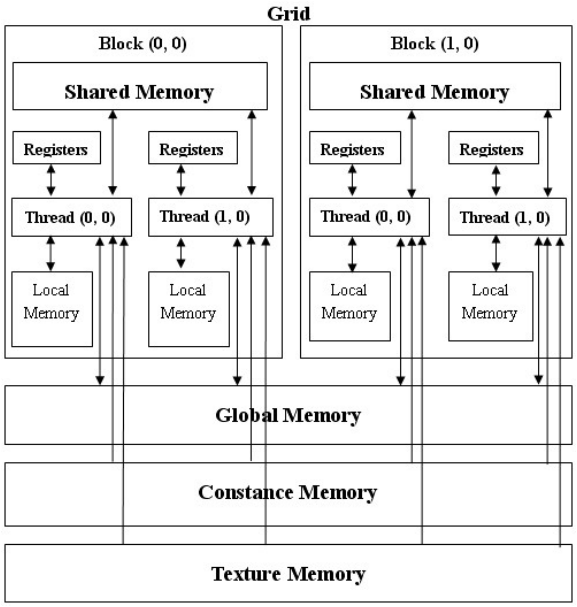


Figure 1. Three-level thread model of CUDA

In order to make full use of the computing resources and enhance the adaptive capacity of the algorithms, it is necessary to take advantage of GPU allocated hardware resources as well as considering the instructions optimizing problems.

1) Achieve maximization of hardware resources:

Under the premise of knowing the number of the running blocks and threads of per streaming processor, we can first determine the size of block, and then calculate the count of a block by the scale of the problems and available hardware resources. The division principles of blocks and the threads in the block are related to the use of memory capacity of the smallest sub-problem and the number of them. Generically, the use of memory capacity of sub-problem can be one of the following situations:

a) When the use of memory capacity of sub- problem is small enough that the shared memory of a thread module can contain several sub-problems' data, it is better to make one block process parallel neighboring sub-problems or combine several sub-problems to a bigger one.

b) When the shared memory of a thread block is only enough to contain one sub-problem's data, it could simply transfer data to the kernel function and calculate the result.

c) When the shared memory of the block is smaller than the sub-problem's data, it could use the constant memory and the global memory, or split sub-problems to several procedures which can be processed sequentially and calculate the result by using several kernel functions.

2) Achieve optimization of instruction:

CUDA uses the Single Instruction Multiple Data structure (SIMT). Each processor executes one warp instructions (including 32 threads). Its working principle is as follows: read instruction operands for each thread of the warp at first, then execute the instructions, and finally write the results for each thread. So we can improve throughput of instruction streams by the rules:

a) It's highly recommended to implement calculations with single precision floating-point unit for kernel function. And the accuracy could be improved with the use of Kahan's Summation Formula.

b) It's better to replace local memory by registers or shared memory, and make the shared memory take the place of global memory. And there is a need to pay attention to access latency of shared memory caused by bank conflicts.

IV. EXPERIMENTAL RESULTS

To verify the feasibility of applying GPU to parallel computation on Map Algebra operators, we choose to realize the SUM operator of Local Cell Statistics functions on a NVIDIA primary hardware platform.

A. Experimental Environment

Hardware platform: CPU is AMD 3600 + dual-core processor(frequency of 1.9GHz), GPU is NVIDIA's GeForce 9500GT(memory capacity is 128MB, memory frequency is 710.0MHz). Software platform: under VS2005 IDE platform, using CUDA 2.3 development toolkit.

B. Experimental Data

Select five groups of different sizes of Dem data whose range from small to big are 460*320, 460*640, 920*640, 925*1220 and 2775*3660.

C. Experimental Procedures

1) Data Organization:

Suppose there are 2-Layer data file of the same resolution (file A and file B, whose size are m * n). Read them into float Array[2][m*n].

2) Computational Experiments of CPU:

Operate arrays by for loop and save results to the res[m*n], res[i]= Array[0][i]+ Array[1][i].

3) Computational Experiments of GPU:

a) Determine the size of block according to the current hardware conditions. Then define the number of block by the scope of data. Since 512 threads per block is the maximum limit currently, it is better to have about 256 threads per block doing parallel computation. And the number of block can be obtained by the following formula: the data quantity of file/the thread number of per block.

b) Determine the size of data block and the element number in it by capacity of CUDA global memory and size of the data as follows:

*Int EleNum=MaxMemoryPitch/(Z*sizeof(float));*

*Int BlkNum=(m*n+EleNum-1)/EleNum;*

MaxMemoryPitch is the maximum pitch allowed by the memory copy functions of CUDA.

c) Allocate the parallel computation and result space in video memory by using CUDA functions. Then transfer 2-layer data from host to device and implement the following kernel function:

```
__global__ static void SumKernel
(float* arr_d, size_t pitch, float* arry, int Z)
{
  float data,Data=0.0f;
  for(int i=0; i<Z; i++)
  {
    float*row=(float*)((char*)arr_d + pitch*i);
    data=row[blockIdx.x*blockDim.x+threadIdx.x];
    Data+=data;
  }
  arry[blockIdx.x*blockDim.x+threadIdx.x]=Data;
}
```

d) Transfer computational results from video memory to host memory by using CUDA function and save the results to Dem data file.

4) Experimental Results and Analysis:

We have got the following results for five groups of datasets. The results show that GPU presents a great advantage in large amounts of data operations. When the amount of data is greater than 2775*3660, GPU computational speed on the same data is significantly increased in comparison with CPU. But when the amount of data is small, GPU parallel processing ability is less obvious than that of CPU for the data transmission between the host memory and the video memory would cost a lot of time.

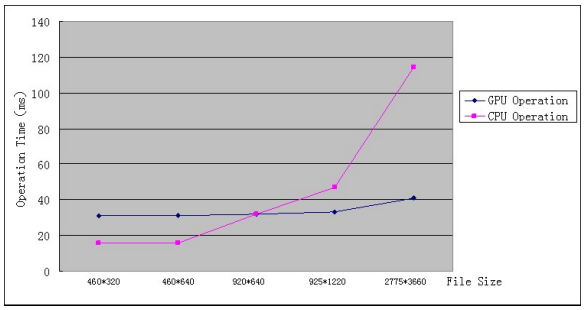


Figure 2. GPU and CPU's Comparison of SUM Operator

V. CONCLUSIONS AND FUTURE WORK

In this paper we have presented methods for using GPU parallel processing architecture for accelerating algorithm execution of map algebra. It could provide a new solution for optimized research of spatial analytic algorithms by means of combining the mature theory of map algebra with the high-efficiency parallel computational resources nowadays.

As future work, we would like to design a new parallel raster data storage model matching for the parallel algorithms on GPU and find a memory scheduling algorithm suitable for the new data parallel access patterns.

ACKNOWLEDGMENT

The authors would like to thank Wenxin Yang for implementing and optimizing CUDA code of SUM operators. We also thank Jing Sun and Yonghong Lv for collection work on algorithms of map algebra in their winter vacation. The web site of http://www.gpgpu.org and http://en.mimi.hu/gis were very useful.

REFERENCES

[1] Tomlin,C.D. *Geographic systems and cartographic modeling*, Englewood Cliffs, NJ, USA: Prentice-Hall, 1990.

[2] Pullar,D. *MapScript: A map algebra programming language incorporating neighborhood analysis*, GeoInformatica, 2001, 5(2), pp.145-163.

[3] Ostlander,N. *Interoperable Services for Web-Based Spatial Decision Support* 7th AGILE Conference on Geographic Information Science, Heraklion, Greece, AGILE, 2004.

[4] Mennis,J., Viger,R., and Tomlin,C.D. *Cubic map algebra functions for spatio-temporal analysis*, Cartography and Geographical Information Science, 2005, 32(1), pp.17-32.

[5] Frank,A. *Map Algebra Extended with Functors for Temporal Data.Perspectives in Conceptual Modeling*, ER 2005 Workshops, Klagenfurt, Austria, Springer, 2005.

[6] Gilberto,C., Danilo,P.and Ricardo,C,etc. *Towards a generalized map algebra: principles and data types*, Brazil, 2007.

[7] Yacine Amara, Xavier Marsault. *A GPU Tile-Load-Map architecture for terrain rendering:theory and applications*, Vis Comput, 2009, vol.25, pp.805-824.

[8] Eladio Gutierrez,Sergio Romero,Maria A.Trenas,etc. *Experiences with Mapping Non-linear Memory Access Patterns into GPUs*, ICCS 2009, Springer-Verlag Berlin Heidelberg, pp.924-933.

[9] S.S.Stone,J.P.Haldar,S.C.Tsao,etc. *Accelerating advanced MRI on GPUs*, Parallel Distrib.Comput, 2008, vol.68, pp.1307-1318.

[10] Weiguo Liu,Bertil Schmidt,Gerrit Voss,etc. *Accelerating molecular Dynamics simulations using Graphics Processing Units with CUDA*, Computer Physics Communications, 2008, vol.179, pp.634-641.